10-빌드-및-배포

개요

이 문서는 Monad 블록체인 실행 엔진의 빌드 시스템, 컴파일 설정, 의존성 관리, 그리고 배포 전략을 상세히 분석합니다. Monad는 CMake 기반의 빌드 시스템을 사용하며, 고성능을 위해 특정 CPU 아키텍처와 컴파일러 최적화를 요구합니다.

주요 내용

1. **빌드 시스템**: CMake 기반 빌드 구조 및 컴포넌트 구성
2. **컴파일 플래그**: CPU 요구사항 및 최적화 설정
3. **의존성 관리**: Git 서브모듈 및 시스템 패키지
4. **배포 전략**: Docker 기반 빌드 및 바이너리 배포

관련 파일

- `CMakeLists.txt` : 루트 빌드 설정
- `category/*/CMakeLists.txt` : 각 컴포넌트별 빌드 설정
- `scripts/configure.sh` , `scripts/build.sh` : 빌드 스크립트
- `docker/Dockerfile` : Docker 빌드 설정
- `.gitmodules` : Git 서브모듈 의존성
- `category/core/toolchains/*.cmake` : CPU별 툴체인 설정

---

1. 빌드 시스템 구조

1.1 CMake 기반 빌드 시스템

Monad는 CMake 3.27 이상을 요구하며, Ninja 빌드 시스템을 기본으로 사용합니다.

루트 CMakeLists.txt 구조

```
# CMakeLists.txt
cmake_minimum_required(VERSION 3.27)
project(monad)

# 컴파일러 버전 요구사항
if(CMAKE_CXX_COMPILER_ID STREQUAL "GNU")
if(CMAKE_CXX_COMPILER_VERSION VERSION_LESS 15)
message(FATAL_ERROR "GCC version 15 or higher is required.")
endif()
elseif(CMAKE_CXX_COMPILER_ID STREQUAL "Clang")
if(CMAKE_CXX_COMPILER_VERSION VERSION_LESS 19)
message(FATAL_ERROR "Clang version 19 or higher is required.")
endif()
endif()
```

컴파일러 요구사항:

- GCC 15 이상
- Clang 19 이상
- libstdc++ 표준 라이브러리 (libc++는 미지원)

빌드 옵션

```
# 주요 빌드 옵션
option(MONAD_COMPILER_LLVM "Build llvm backend" OFF)
option(MONAD_COMPILER_COVERAGE "Build with coverage" OFF)
option(MONAD_COMPILER_TESTING "Build compiler tests" OFF)
option(MONAD_COMPILER_BENCHMARKS "Build compiler benchmarks" OFF)
option(MONAD_COMPILER_DUMP_ASM "Dump assembly files into build/asm" OFF)
option(MONAD_COMPILER_STATS "Print statistics about the compiler" OFF)
option(MONAD_COMPILER_HOT_PATH_STATS "Print statistics about the vm" OFF)
option(BUILD_SHARED_LIBS "Build monad with shared libraries" OFF)
```

옵션 설명:

- `MONAD_COMPILER_LLVM` : LLVM 백엔드 활성화 (기본: OFF)
- `MONAD_COMPILER_TESTING` : 컴파일러 테스트 빌드
- `MONAD_COMPILER_BENCHMARKS` : 벤치마크 빌드
- `BUILD_SHARED_LIBS` : 공유 라이브러리 빌드 (기본: OFF, 정적 링크)

공통 컴파일 옵션 함수

```
function(monad_compile_options target)
# C/C++ 표준 설정
set_property(TARGET ${target} PROPERTY C_STANDARD 23)
set_property(TARGET ${target} PROPERTY CXX_STANDARD 23)
set_property(TARGET ${target} PROPERTY CXX_STANDARD_REQUIRED ON)
```

```
# 경고 옵션
target_compile_options(${target} PRIVATE -Wall -Wextra -Wconversion -Werror)
# GNU 소스 정의
target_compile_definitions(${target} PUBLIC "_GNU_SOURCE")
# Quill 로거 설정
target_compile_definitions(${target} PUBLIC QUILL_ROOT_LOGGER_ONLY)
# nlohmann_json ranges 지원 비활성화 (Clang 버그 회피)
target_compile_definitions(${target} PUBLIC "JSON_HAS_RANGES=0")
endfunction()
```

특징:

- C23, C++23 표준 사용
- 모든 경고를 에러로 처리 (`-Werror`)
- GNU 확장 기능 활성화
- 컴파일러별 특정 경고 비활성화

1.2 컴포넌트별 빌드 구조

Monad는 모듈화된 구조로, 각 컴포넌트가 독립적인 CMakeLists.txt를 가집니다.

컴포넌트 계층 구조

```
monad_execution (최종 라이브러리)
├── monad_core (category/core)
```

```
├── monad_async (category/async)
├── monad_trie (category/mpt)
├── monad_execution_ethereum (category/execution/ethereum)
├── monad_execution_native (category/execution/monad)
├── monad_rpc (category/rpc)
├── monad_statesync (category/statesync)
└── monad-vm (category/vm)
├── monad-vm-core
├── monad-vm-compiler
├── monad-vm-evm
├── monad-vm-interpreter
├── monad-vm-runtime
└── monad-vm-utils
```

VM 컴포넌트 빌드 예시

```
# category/vm/CMakeLists.txt
add_library(monad-vm OBJECT)

target_sources(monad-vm PRIVATE
"code.hpp"
"compiler.cpp"
"compiler.hpp"
"varcode_cache.cpp"
```

```
"varcode_cache.hpp"

"vm.cpp"

"vm.hpp"

)


monad_compile_options(monad-vm)


target_link_libraries(monad-vm

PUBLIC TBB::tbb

PUBLIC intx::intx

PUBLIC evmc::evmc

PUBLIC asmjit::asmjit

PUBLIC quill::quill

PUBLIC monad-vm::monad-vm-compiler

PUBLIC monad-vm::monad-vm-interpreter

PRIVATE monad-vm::monad-vm-core

PRIVATE monad-vm::monad-vm-runtime

PRIVATE monad-vm::monad-vm-utils

)
```

특징:

- OBJECT 라이브러리 사용 (중간 오브젝트 파일)
- PUBLIC/PRIVATE 링크 구분으로 의존성 전파 제어

- 네임스페이스 별칭 (`monad-vm::monad-vm`)

MPT/TrieDB 컴포넌트 빌드 예시

```
# category/mpt/CMakeLists.txt
add_library(monad_trie OBJECT
"compute.cpp"
"db.cpp"
"find.cpp"
"node.cpp"
"trie.cpp"
"update_aux.cpp"
# ... 기타 소스 파일
)

target_link_libraries(monad_trie
PRIVATE Boost::boost
PUBLIC concurrentqueue
PUBLIC monad_async
PUBLIC quill::quill
)

# CLI 도구
add_executable(monad_mpt "cli_tool_main.cpp" "cli_tool_impl.cpp")
```

```
target_link_libraries(monad_mpt
PUBLIC monad_execution
PkgConfig::zstd
PkgConfig::archive
CLI11::CLI11
)
```

최종 실행 라이브러리 조립

```
# CMakeLists.txt
add_library(monad_execution)
target_link_libraries(
monad_execution
PUBLIC monad_core
monad_async
monad_trie
monad_execution_ethereum
monad_execution_native
monad_rpc
monad_statesync
monad-vm
monad-vm-core
monad-vm-compiler
monad-vm-evm
```

```
monad-vm-interpreter
monad-vm-runtime
monad-vm-utils)
```

빌드 흐름:

1. 각 컴포넌트를 OBJECT 라이브러리로 빌드
2. `monad_execution` 라이브러리로 통합
3. 실행 파일 (`monad` , `monad_cli` , `monad_mpt`)에 링크

1.3 테스트 빌드 시스템

단위 테스트 함수

```
function(monad_add_test2 target)
add_executable(
${target}
"${CMAKE_CURRENT_FUNCTION_LIST_DIR}/test/unit/common/src/test/main.cpp"
${ARGN})
monad_compile_options(${target})
target_link_libraries(${target}
monad_execution
GTest::GTest
GTest::Main)
gtest_discover_tests(
${target}
```

```
DISCOVERY_MODE PRE_TEST

PROPERTIES

ENVIRONMENT ASAN_OPTIONS=abort_on_error=1

ENVIRONMENT UBSAN_OPTIONS=halt_on_error=1,print_stacktrace=1

ENVIRONMENT TSAN_OPTIONS=external_symbolizer_path=/usr/bin/llvm-symbolizer)

endfunction()
```

특징:

- Google Test 프레임워크 사용
- Sanitizer 환경 변수 자동 설정
- 테스트 자동 발견 (gtest_discover_tests)

빌드 스크립트

[configure.sh](#):

```
#!/bin/bash

cmake \

-DCMAKE_EXPORT_COMPILE_COMMANDS:BOOL=TRUE \

-DCMAKE_BUILD_TYPE:STRING=${CMAKE_BUILD_TYPE:-RelWithDebInfo} \

-B build \

-G  Ninja
```

[build.sh](#):

```
#!/bin/bash
```

```
VERBOSE=1  \
cmake \
--build build \
--config ${CMAKE_BUILD_TYPE:-RelWithDebInfo} \
--target all
```

기본 빌드 타입:

- `RelWithDebInfo` : 최적화 + 디버그 심볼 (기본값)
- `Release` : 최대 최적화
- `Debug` : 디버그 전용

---

2. 컴파일 플래그 및 CPU 요구사항

2.1 CPU 아키텍처 요구사항

Monad는 고성능 암호화 연산을 위해 최신 CPU 명령어 세트를 요구합니다.

x86-64-v3 ISA 레벨

최소 요구사항: x86-64-v3 (Haswell 이상)

x86-64-v3는 다음 명령어 세트를 포함합니다:

- **AVX2**: 256비트 SIMD 연산
- **BMI1/BMI2**: 비트 조작 명령어
- **FMA**: Fused Multiply-Add
- **MOVBE**: 바이트 순서 변환

- **LZCNT**: Leading Zero Count

지원 CPU:

- Intel: Haswell (2013) 이상
- AMD: Excavator (2015) 이상

툴체인 설정 파일

gcc-avx2.cmake (Haswell):

```
# category/core/toolchains/gcc-avx2.cmake

set(CMAKE_ASM_FLAGS_INIT "-march=haswell")

set(CMAKE_C_FLAGS_INIT "-march=haswell")

set(CMAKE_CXX_FLAGS_INIT "-march=haswell")
```

gcc-avx512.cmake (Skylake-AVX512):

```
# category/core/toolchains/gcc-avx512.cmake

set(CMAKE_ASM_FLAGS_INIT "-march=skylake-avx512")

set(CMAKE_C_FLAGS_INIT "-march=skylake-avx512")

set(CMAKE_CXX_FLAGS_INIT "-march=skylake-avx512")
```

사용 방법:

```
# Haswell (AVX2)

CMAKE_TOOLCHAIN_FILE=category/core/toolchains/gcc-avx2.cmake  ./scripts/configur



# Skylake-AVX512
```

```
CMAKE_TOOLCHAIN_FILE=category/core/toolchains/gcc-avx512.cmake  ./scripts/config


# 로컬 CPU 최적화 (권장: 개발 환경)
CFLAGS="-march=native"  CXXFLAGS="-march=native"  ASMFLAGS="-march=native"  ./sc
```

2.2 최적화 레벨

CMAKE_BUILD_TYPE 옵션

RelWithDebInfo (기본값):

```
CMAKE_BUILD_TYPE=RelWithDebInfo
# 컴파일러 플래그: -O2 -g
```

- 최적화 레벨 2
- 디버그 심볼 포함
- 프로파일링 가능
- 프로덕션 성능에 근접

Release:

```
CMAKE_BUILD_TYPE=Release
# 컴파일러 플래그: -O3 -DNDEBUG
```

- 최대 최적화 레벨 3
- assert 비활성화

- 디버그 심볼 없음
- 최고 성능

Debug:

```
CMAKE_BUILD_TYPE=Debug
# 컴파일러 플래그: -O0 -g
```

- 최적화 없음
- 전체 디버그 심볼
- 디버거 사용 최적
- 성능 저하

빌드 타입별 사용 사례

빌드 타입	용도	성능	디버깅
Debug	개발, 디버깅	낮음	최상
RelWithDebInfo	프로파일링, 테스트	높음	양호
Release	프로덕션 배포	최고	제한적

2.3 컴파일러별 최적화

GCC 15 최적화

```
# GCC 전용 경고 비활성화
target_compile_options(${target}
PRIVATE $<$<CXX_COMPILER_ID:GNU>:-Wno-missing-field-initializers>)
```

```
# Release 빌드 시 특정 경고 비활성화
target_compile_options(quill
PUBLIC $<$<AND:$<CXX_COMPILER_ID:GNU>,$<CONFIG:Release>>:-Wno-stringop-overflow>
```

GCC 특징:

- 더 나은 벡터화
- 링크 타임 최적화 (LTO) 지원
- 프로파일 기반 최적화 (PGO) 가능

Clang 19 최적화

```
# Clang 전용 속성 경고 비활성화
target_compile_options(${target}
PUBLIC $<$<CXX_COMPILER_ID:GNU>:-Wno-attributes=clang::no_sanitize>)
```

Clang 특징:

- 더 빠른 컴파일 속도
- 더 나은 에러 메시지
- Sanitizer 지원 우수

2.4 Sanitizer 빌드

Address Sanitizer (ASAN)

```
# category/core/toolchains/gcc-asan.cmake
set(CMAKE_C_FLAGS_INIT "-fsanitize=address -fno-omit-frame-pointer")
```

```
set(CMAKE_CXX_FLAGS_INIT "-fsanitize=address -fno-omit-frame-pointer")
```

용도: 메모리 오류 탐지 (버퍼 오버플로우, use-after-free 등)

Thread Sanitizer (TSAN)

```
# category/core/toolchains/gcc-tsan.cmake

set(CMAKE_C_FLAGS_INIT "-fsanitize=thread")

set(CMAKE_CXX_FLAGS_INIT "-fsanitize=thread")
```

용도: 데이터 레이스 및 동시성 버그 탐지

Fuzzing 빌드

```
# category/core/toolchains/clang-fuzz.cmake

set(CMAKE_C_FLAGS_INIT "-fsanitize=fuzzer-no-link,address")

set(CMAKE_CXX_FLAGS_INIT "-fsanitize=fuzzer-no-link,address")
```

용도: 퍼징 테스트 (libFuzzer)

2.5 성능 최적화 플래그 비교

-march 플래그 비교

| 플래그 | 대상 CPU | 호환성 | 성능 | 권장 용도 |

|--------|---------|--------|------|----------|

| `-march=x86-64-v3` | 2013년 이후 | 넓음 | 양호 | 최소 요구사항 |

| `-march=haswell` | Haswell 이상 | 넓음 | 양호 | 배포용 (Docker) |

| `-march=skylake-avx512` | Skylake 이상 | 중간 | 우수 | AVX-512 서버 |

| `-march=native` | 현재 CPU | 없음 | 최고 | 로컬 개발 |

실제 빌드 명령어 예시

배포용 빌드 (Haswell):

```
CC=gcc-15  CXX=g++-15  \
CFLAGS="-march=haswell" \
CXXFLAGS="-march=haswell" \
ASMFLAGS="-march=haswell" \
CMAKE_BUILD_TYPE=Release \
./scripts/configure.sh && ./scripts/build.sh
```

개발용 빌드 (Native):

```
CC=gcc-15  CXX=g++-15  \
CFLAGS="-march=native" \
CXXFLAGS="-march=native" \
ASMFLAGS="-march=native" \
CMAKE_BUILD_TYPE=RelWithDebInfo \
./scripts/configure.sh && ./scripts/build.sh
```

디버그 빌드:

```
CC=gcc-15  CXX=g++-15  \
CFLAGS="-march=haswell" \
```

```
CXXFLAGS="-march=haswell" \
ASMFLAGS="-march=haswell" \
CMAKE_BUILD_TYPE=Debug \
./scripts/configure.sh && ./scripts/build.sh
```

LLVM 백엔드 포함 빌드:

```
CC=gcc-15  CXX=g++-15  \
CFLAGS="-march=haswell" \
CXXFLAGS="-march=haswell" \
ASMFLAGS="-march=haswell" \
./scripts/configure.sh
cmake  build  -DMONAD_COMPILER_LLVM=On
./scripts/build.sh
```

---

3. 의존성 관리

Monad는 두 가지 유형의 의존성을 사용합니다:

1. **Self-managed**: Git 서브모듈로 관리되는 의존성
2. **System**: 시스템 패키지 매니저로 설치되는 의존성

3.1 Self-managed 의존성 (Git Submodules)

서브모듈 초기화

```
git submodule update --init --recursive
```

주요 서브모듈 목록

```
# .gitmodules
[submodule "third_party/asmjit"]
path = third_party/asmjit
url = https://github.com/asmjit/asmjit.git

[submodule "third_party/BLAKE3"]
path = third_party/BLAKE3
url = https://github.com/BLAKE3-team/BLAKE3.git

[submodule "third_party/blst"]
path = third_party/blst
url = https://github.com/supranational/blst.git

[submodule "third_party/c-kzg-4844"]
path = third_party/c-kzg-4844
url = https://github.com/ethereum/c-kzg-4844.git

[submodule "third_party/concurrentqueue"]
path = third_party/concurrentqueue
```

```
url = https://github.com/cameron314/concurrentqueue


[submodule "third_party/cthash"]
path = third_party/cthash
url = https://github.com/hanickadot/cthash.git


[submodule "third_party/ethash"]
path = third_party/ethash
url = https://github.com/chfast/ethash.git


[submodule "third_party/ethereum-tests"]
path = third_party/ethereum-tests
url = https://github.com/ethereum/tests


[submodule "third_party/evmc"]
path = third_party/evmc
url = https://github.com/ipsilon/evmc.git


[submodule "third_party/intx"]
path = third_party/intx
url = https://github.com/chfast/intx.git


[submodule "third_party/komihash"]
```

```
path = third_party/komihash
url = https://github.com/avaneev/komihash.git

[submodule "third_party/nlohmann_json"]
path = third_party/nlohmann_json
url = https://github.com/nlohmann/json.git

[submodule "third_party/quill"]
path = third_party/quill
url = https://github.com/odygrd/quill.git

[submodule "third_party/silkpre"]
path = third_party/silkpre
url = https://github.com/category-labs/silkpre.git

[submodule "third_party/unordered_dense"]
path = third_party/unordered_dense
url = https://github.com/martinus/unordered_dense.git
```

서브모듈 의존성 상세

| 라이브러리 | 용도 | 버전 관리 |

|---|---|---|

| **asmjit** | JIT 어셈블러 (x86 코드 생성) | Git 서브모듈 |

| **BLAKE3** | 고속 암호화 해시 함수 | Git 서브모듈 |

| **blst** | BLS12-381 서명 라이브러리 | Git 서브모듈 |

| **c-kzg-4844** | KZG 다항식 커밋먼트 | Git 서브모듈 |

| **concurrentqueue** | 락프리 동시성 큐 | Git 서브모듈 |

| **cthash** | 컴파일 타임 해시 | Git 서브모듈 |

| **ethash** | Ethash PoW 알고리즘 | Git 서브모듈 |

| **ethereum-tests** | 이더리움 공식 테스트 스위트 | Git 서브모듈 |

| **evmc** | Ethereum VM C API | Git 서브모듈 |

| **intx** | 256비트 정수 연산 | Git 서브모듈 |

| **komihash** | 고속 해시 함수 | Git 서브모듈 |

| **nlohmann_json** | JSON 파싱 라이브러리 | Git 서브모듈 |

| **quill** | 비동기 로깅 라이브러리 | Git 서브모듈 |

| **silkpre** | Precompile 구현 | Git 서브모듈 |

| **unordered_dense** | 고성능 해시맵 | Git 서브모듈 |

CMake 통합 방식

```
# CMakeLists.txt


# asmjit
set(ASMJIT_STATIC ON)
add_subdirectory(third_party/asmjit)


# BLAKE3
```

```
add_subdirectory(third_party/BLAKE3/c)


# blst (커스텀 빌드 스크립트)
set(DOWNLOAD_BLST OFF)
include(cmake/blst.cmake)


# c-kzg-4844
add_subdirectory("third_party/c-kzg-4844-builder")


# concurrentqueue
add_subdirectory("third_party/concurrentqueue")


# ethash
set(ETHASH_TESTING NO)
add_subdirectory("third_party/ethash")


# intx
add_subdirectory("third_party/intx")


# evmc
set(HUNTER_ENABLED OFF)
add_subdirectory("third_party/evmc")
```

```
# nlohmann_json
add_subdirectory("third_party/nlohmann_json" SYSTEM)


# quill
add_subdirectory("third_party/quill")


# silkpre
set(OPTIONAL_BUILD_TESTS OFF)
add_subdirectory(third_party/silkpre)


# unordered_dense
add_subdirectory("third_party/unordered_dense")
```

특징:

- 각 라이브러리의 빌드 옵션 제어
- 테스트 비활성화로 빌드 시간 단축
- SYSTEM 플래그로 경고 억제

3.2 System 의존성 (APT 패키지)

필수 시스템 패키지

개발 도구:

```
# scripts/ubuntu-build/install-tools.sh
packages=(
```

```
apt-utils
ca-certificates
clang-19
clang-tools-19
clang-tidy-19
cmake
curl
g++-15
gcc-15
gdb
git
gnupg
ninja-build
pkg-config
python-is-python3
python3-pytest
software-properties-common
valgrind
wget
)
```

라이브러리 의존성:

```
# scripts/ubuntu-build/install-deps.sh
packages=(
libarchive-dev  # 아카이브 압축/해제
libbenchmark-dev  # Google Benchmark
libbrotli-dev  # Brotli 압축
libcap-dev  # POSIX capabilities
libcgroup-dev  # cgroup 관리
libcli11-dev  # CLI 파싱
libcrypto++-dev  # 암호화 라이브러리
libgmock-dev  # Google Mock
libgmp-dev  # 다정밀도 정수 연산
libgtest-dev  # Google Test
libhugetlbfs-dev  # Huge pages 지원
libtbb-dev  # Intel TBB
liburing-dev  # io_uring 라이브러리
libzstd-dev  # Zstandard 압축
)
```

Boost 라이브러리:

```
# scripts/ubuntu-build/install-boost.sh
apt-get  install  -y \
libboost-fiber1.83.0 \
```

```
libboost-json1.83.0 \
libboost-stacktrace1.83.0 \
libboost-fiber1.83-dev \
libboost-json1.83-dev \
libboost-stacktrace1.83-dev \
libboost1.83-dev
```

Boost 버전 요구사항: 1.83 이상

CMake 패키지 검색

```
# CMakeLists.txt

# Boost (CONFIG 모드)
find_package(Boost REQUIRED COMPONENTS fiber json CONFIG)

# PkgConfig 기반 패키지
find_package(PkgConfig REQUIRED)
pkg_check_modules(brotli REQUIRED IMPORTED_TARGET libbrotlienc libbrotlidec)
pkg_check_modules(crypto++ REQUIRED IMPORTED_TARGET libcrypto++)

# CLI11
find_package(CLI11 REQUIRED)

# TBB
```

```
find_package(TBB REQUIRED)


# Google Test
find_package(GTest)


# LLVM (선택적)
if(MONAD_COMPILER_LLVM)
find_package(LLVM 19.1.7 REQUIRED CONFIG)
endif()
```

의존성 버전 요구사항 정리

패키지	최소 버전	용도
GCC	15	C/C++ 컴파일러
Clang	19	C/C++ 컴파일러 (대안)
CMake	3.27	빌드 시스템
Boost	1.83	Fiber, JSON, Stacktrace
LLVM	19.1.7	LLVM 백엔드 (선택적)
liburing	-	io_uring 지원
TBB	-	병렬 처리

3.3 헤더 전용 라이브러리

일부 라이브러리는 헤더 전용으로 직접 포함됩니다:

```
# ankerl unordered_map
add_library(ankerl_hash INTERFACE)
target_include_directories(ankerl_hash INTERFACE "third_party/ankerl")


# komihash
add_library(komihash INTERFACE)
target_include_directories(komihash INTERFACE "third_party/komihash")


# thread-safe-lru
add_library(thread_safe_lru INTERFACE)
target_include_directories(thread_safe_lru INTERFACE "third_party/thread-safe-lr
```

특징:

- 컴파일 불필요
- 빠른 통합
- 템플릿 기반 구현

---

4. 배포 전략

4.1 Docker 기반 빌드

Monad는 멀티스테이지 Docker 빌드를 사용하여 재현 가능한 빌드 환경을 제공합니다.

Dockerfile 구조

```
# docker/Dockerfile


# 베이스 이미지: Ubuntu 25.04
FROM ubuntu:25.04 AS base


RUN apt-get update && apt-get upgrade -y


# 빌드 스크립트 복사
COPY scripts/ubuntu-build/ /opt


# 의존성 설치
RUN /opt/install-boost.sh
RUN /opt/install-tools.sh
RUN /opt/install-deps.sh
```

빌드 및 테스트 스테이지

```
FROM base AS build_and_test


COPY . src


ARG CC
ARG CXX
```

```
ARG CMAKE_BUILD_TYPE

ARG GIT_COMMIT_HASH


ENV GIT_COMMIT_HASH=$GIT_COMMIT_HASH


# 설정 및 빌드

RUN cd src && \

CC=${CC:?} CXX=${CXX:?} CMAKE_BUILD_TYPE=${CMAKE_BUILD_TYPE:?} \

CFLAGS="-march=haswell" \

CXXFLAGS="-march=haswell" \

ASMFLAGS="-march=haswell" \

./scripts/configure.sh


RUN cd src && ./scripts/build.sh


# 테스트 실행 (io_uring 사용을 위해 security=insecure 필요)

RUN --security=insecure cd src && \

CC=${CC:?} CXX=${CXX:?} CMAKE_BUILD_TYPE=${CMAKE_BUILD_TYPE:?} \

./scripts/test.sh
```

특징:

- Haswell 타겟으로 최대 호환성
- 빌드 인자로 컴파일러 선택 가능

- Git 커밋 해시 임베딩
- io_uring 테스트를 위한 특수 권한

VM 테스트 스테이지

```
FROM base AS build_and_test_vm

COPY . src

ARG CC
ARG CXX
ARG CMAKE_BUILD_TYPE
ARG SKIP_LLVM_TESTS
ARG TOOLCHAIN

# LLVM 백엔드 포함 빌드
RUN cd src && \
CC=${CC:?} CXX=${CXX:?} CMAKE_BUILD_TYPE=${CMAKE_BUILD_TYPE:?} \
CMAKE_TOOLCHAIN_FILE=category/core/toolchains/${TOOLCHAIN:?}.cmake \
./scripts/configure.sh

RUN cd src && \
cmake build -DMONAD_COMPILER_TESTING=On -DMONAD_COMPILER_LLVM=On
```

```
RUN cd src && ./scripts/vm/build-tests.sh

RUN cd src && SKIP_LLVM_TESTS=${SKIP_LLVM_TESTS:?} ./scripts/vm/test.sh
```

코드 품질 검사 스테이지

```
FROM base AS code_quality


COPY . src


RUN cd src && \

CMAKE_TOOLCHAIN_FILE=category/core/toolchains/gcc-avx2.cmake \

cmake -S . -B build \

-DMONAD_COMPILER_BENCHMARKS=On \

-DMONAD_COMPILER_LLVM=On \

-DMONAD_COMPILER_TESTING=On \

-DCMAKE_BUILD_TYPE=Debug \

-DCMAKE_C_COMPILER=clang-19 \

-DCMAKE_CXX_COMPILER=clang++-19


RUN cd src && ./scripts/check-clang-tidy.sh
```

퍼징 스테이지

```
FROM base AS vm_fuzz
```

```
RUN apt-get update && apt-get install -y tmux

COPY . src

RUN cd src && \
CMAKE_TOOLCHAIN_FILE=category/core/toolchains/gcc-avx2.cmake \
cmake -S . -B build \
-GNinja \
-DCMAKE_BUILD_TYPE=Release \
-DMONAD_COMPILER_LLVM=Off \
-DMONAD_COMPILER_TESTING=On \
-DCMAKE_C_COMPILER=clang-19 \
-DCMAKE_CXX_COMPILER=clang++-19

RUN cd src && cmake --build build -t monad-compiler-fuzzer
RUN cd src && ./scripts/vm/ci-fuzzer.sh
```

런타임 이미지

```
FROM base AS runner

# 빌드된 바이너리만 복사
COPY --from=build_and_test /src/build/category/mpt/monad_mpt /usr/local/bin/
```

```
COPY --from=build_and_test /src/build/cmd/monad_cli /usr/local/bin/

COPY --from=build_and_test /src/build/cmd/monad /usr/local/bin/
```

특징:

- 최소 런타임 이미지
- 빌드 아티팩트만 포함
- 작은 이미지 크기

Docker 빌드 명령어

기본 빌드:

```
docker  build \

--target build_and_test \

--build-arg  CC=gcc-15 \

--build-arg CXX=g++-15 \

--build-arg  CMAKE_BUILD_TYPE=Release \

--build-arg GIT_COMMIT_HASH=$(git rev-parse HEAD) \

-f  docker/Dockerfile \

-t monad-execution:latest \

.
```

VM 테스트 빌드:

```
docker  build \

--target build_and_test_vm \
```

```
--build-arg  CC=gcc-15 \
--build-arg CXX=g++-15 \
--build-arg  CMAKE_BUILD_TYPE=Release \
--build-arg TOOLCHAIN=gcc-avx2 \
--build-arg  SKIP_LLVM_TESTS=0 \
-f docker/Dockerfile \
-t  monad-vm-test:latest \
.
```

런타임 이미지:

```
docker  build \
--target runner \
--build-arg  CC=gcc-15 \
--build-arg CXX=g++-15 \
--build-arg  CMAKE_BUILD_TYPE=Release \
-f docker/Dockerfile \
-t  monad-runtime:latest \
.
```

4.2 바이너리 배포

빌드 출력물

빌드 완료 후 다음 바이너리가 생성됩니다:

```
build/
├── cmd/
| ├── monad # 메인 실행 데몬
| └── monad_cli # CLI 도구
├── category/mpt/
| └── monad_mpt # TrieDB CLI 도구
└── test/
├── ethereum_test/ # 이더리움 테스트 실행기
└── unit/ # 단위 테스트들
```

주요 바이너리 설명

monad:

- Monad 블록체인 실행 데몬
- 합의 컴포넌트와 통신
- 블록 실행 및 상태 관리
- 사용 예시:

```
./build/cmd/monad --config /path/to/config.json
```

monad_cli:

- 명령줄 인터페이스 도구
- 블록 재생 및 검증
- 개발 및 디버깅 용도

monad_mpt:

- TrieDB 관리 도구
- 데이터베이스 검증 및 압축
- 스냅샷 생성 및 복원

설치 방법

시스템 전역 설치:

```
sudo cp build/cmd/monad /usr/local/bin/
sudo cp build/cmd/monad_cli /usr/local/bin/
sudo cp build/category/mpt/monad_mpt /usr/local/bin/
```

사용자 로컬 설치:

```
mkdir -p ~/.local/bin
cp build/cmd/monad ~/.local/bin/
cp build/cmd/monad_cli ~/.local/bin/
cp build/category/mpt/monad_mpt ~/.local/bin/
export PATH="$HOME/.local/bin:$PATH"
```

4.3 설정 파일 관리

실행 설정

Monad 실행 데몬은 JSON 형식의 설정 파일을 사용합니다.

설정 파일 예시 (config.json):

```
{
"execution": {
"ledger_dir": "/data/ledger",
"db_path": "/data/triedb",
"fiber_pool_size": 128,
"compiler_cache_size": "1GB"
},
"network": {
"consensus_endpoint": "localhost:26657"
},
"logging": {
"level": "info",
"file": "/var/log/monad/execution.log"
}
}
```

환경 변수

주요 환경 변수:

```
# 데이터 디렉토리
export MONAD_DATA_DIR=/data/monad

# 로그 레벨
```

```
export MONAD_LOG_LEVEL=debug


# 컴파일러 캐시 크기
export MONAD_COMPILER_CACHE_SIZE=2GB


# Fiber 풀 크기
export MONAD_FIBER_POOL_SIZE=256
```

디렉토리 구조

```
/data/monad/
├── ledger/ # 블록 데이터
| ├── blocks/
| └── receipts/
├── triedb/ # 상태 데이터베이스
| ├── chunks/
| └── metadata/
├── compiler_cache/ # 컴파일된 코드 캐시
└── logs/ # 로그 파일
```

4.4 성능 튜닝

Huge Pages 설정

```
# Huge pages 활성화 (2MB pages)
sudo hugeadm --pool-pages-min 2MB:1024
sudo hugeadm --pool-pages-max 2MB:2048

# 영구 설정
echo "vm.nr_hugepages = 1024" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
```

io_uring 설정

```
# io_uring 큐 크기 증가
echo 4096 | sudo tee /proc/sys/fs/io-uring/max_entries

# 메모리 잠금 제한 증가
ulimit -l unlimited
```

CPU 친화성 설정

```
# 특정 CPU 코어에 바인딩
taskset -c 0-15 ./build/cmd/monad --config config.json

# NUMA 노드 지정
numactl --cpunodebind=0 --membind=0 ./build/cmd/monad --config config.json
```

4.5 배포 체크리스트

빌드 전 확인사항

- [ ] Git 서브모듈 초기화 완료
- [ ] 시스템 의존성 설치 완료
- [ ] 컴파일러 버전 확인 (GCC 15+ 또는 Clang 19+)
- [ ] CMake 버전 확인 (3.27+)
- [ ] CPU 아키텍처 확인 (x86-64-v3 이상)

빌드 확인사항

- [ ] 빌드 타입 선택 (Release/RelWithDebInfo/Debug)
- [ ] CPU 타겟 플래그 설정 (-march=haswell 또는 -march=native)
- [ ] 필요한 빌드 옵션 활성화 (LLVM, Testing 등)
- [ ] 빌드 성공 확인
- [ ] 테스트 실행 및 통과 확인

배포 전 확인사항

- [ ] 바이너리 실행 가능 여부 확인
- [ ] 설정 파일 준비
- [ ] 데이터 디렉토리 생성 및 권한 설정
- [ ] Huge pages 설정 (선택적)
- [ ] io_uring 설정 확인
- [ ] 로그 디렉토리 생성

운영 환경 확인사항

- [ ] CPU 요구사항 충족 (Haswell 이상)
- [ ] 충분한 메모리 (최소 32GB 권장)
- [ ] NVMe SSD 스토리지
- [ ] 네트워크 연결 확인
- [ ] 모니터링 설정

---

5. 빌드 문제 해결

5.1 일반적인 빌드 오류

컴파일러 버전 오류

증상:

```
CMake Error: GCC version 15 or higher is required.
```

해결:

```
# GCC 15 설치
sudo apt-get install gcc-15 g++-15

# 환경 변수 설정
export CC=gcc-15
export CXX=g++-15
```

서브모듈 누락 오류

증상:

```
CMake Error: Could not find third_party/asmjit
```

해결:

```
git submodule update --init --recursive
```

의존성 누락 오류

증상:

```
Could NOT find Boost (missing: fiber json)
```

해결:

```
# Boost 설치
sudo apt-get install libboost-fiber1.83-dev libboost-json1.83-dev

# 또는 전체 의존성 설치
sudo ./scripts/ubuntu-build/install-boost.sh
sudo ./scripts/ubuntu-build/install-deps.sh
```

CPU 명령어 세트 오류

증상:

```
illegal instruction (core dumped)
```

해결:

```
# CPU 기능 확인
lscpu | grep -E "avx2|bmi2|fma"

# 더 낮은 타겟으로 재빌드
CFLAGS="-march=x86-64-v3" CXXFLAGS="-march=x86-64-v3" ./scripts/configure.sh
```

5.2 링크 오류

중복 심볼 오류

증상:

```
multiple definition of `symbol_name`
```

해결:

- OBJECT 라이브러리 사용 확인
- 헤더 가드 확인
- 인라인 함수 정의 확인

미정의 참조 오류

증상:

```
undefined reference to `function_name`
```

해결:

```
# CMakeLists.txt에서 링크 순서 확인
target_link_libraries(target
PUBLIC dependency1
PRIVATE dependency2
)
```

5.3 테스트 실패

io_uring 테스트 실패

증상:

```
io_uring setup failed: Operation not permitted
```

해결:

```
# Docker에서: --security=insecure 플래그 사용
docker build --security=insecure ...

# 로컬에서: 커널 파라미터 확인
cat /proc/sys/kernel/io_uring_disabled
# 0이어야 함
```

메모리 부족 오류

증상:

```
c++: fatal error: Killed signal terminated program cc1plus
```

해결:

```
# 병렬 빌드 제한
cmake  --build  build  -j4  # CPU 코어 수 제한


# 스왑 증가
sudo  fallocate  -l  8G  /swapfile
sudo  chmod  600  /swapfile
sudo  mkswap  /swapfile
sudo  swapon  /swapfile
```

---

6. 빌드 최적화 팁

6.1 빌드 속도 향상

ccache 사용

```
# ccache 설치
sudo  apt-get  install  ccache


# CMake 설정
```

```
export CMAKE_CXX_COMPILER_LAUNCHER=ccache
export CMAKE_C_COMPILER_LAUNCHER=ccache

./scripts/configure.sh
```

Ninja 빌드 시스템

```
# Ninja는 기본적으로 사용됨
# Make보다 빠른 증분 빌드
cmake -G Ninja -B build
```

병렬 빌드

```
# 모든 CPU 코어 사용
cmake --build build -j$(nproc)

# 메모리 부족 시 제한
cmake --build build -j4
```

6.2 디스크 공간 절약

디버그 심볼 제거

```
# Release 빌드 사용
CMAKE_BUILD_TYPE=Release ./scripts/configure.sh
```

```
# 또는 빌드 후 strip
strip build/cmd/monad
```

불필요한 타겟 제외

```
# CMakeLists.txt
option(MONAD_COMPILER_TESTING "Build compiler tests" OFF)
option(MONAD_COMPILER_BENCHMARKS "Build compiler benchmarks" OFF)
```

6.3 개발 워크플로우

증분 빌드

```
# 변경된 파일만 재빌드
cmake --build build

# 특정 타겟만 빌드
cmake --build build --target monad-vm
```

컴파일 명령어 확인

```
# compile_commands.json 생성 (기본 활성화)
cat build/compile_commands.json | jq '.[0]'
```

빌드 시간 측정

```
# 전체 빌드 시간
time cmake --build build

# 타겟별 빌드 시간
cmake  --build  build  --verbose
```

---

7. 요약

7.1 빌드 시스템 핵심 요약

CMake 기반 빌드:

- CMake 3.27 이상 필요
- Ninja 빌드 시스템 사용
- 모듈화된 컴포넌트 구조
- OBJECT 라이브러리로 중간 빌드

컴파일러 요구사항:

- GCC 15 이상 또는 Clang 19 이상
- C23, C++23 표준 사용
- libstdc++ 표준 라이브러리

CPU 요구사항:

- 최소: x86-64-v3 (Haswell)

- 권장: `-march=haswell` (배포용)
- 최적: `-march=native` (개발용)

7.2 의존성 관리 핵심 요약

Self-managed (Git Submodules):

- asmjit, BLAKE3, blst, evmc, intx 등
- `git submodule update --init --recursive` 로 초기화
- CMake 서브프로젝트로 빌드

System (APT 패키지):

- Boost 1.83+, TBB, liburing, GTest 등
- Ubuntu 패키지 매니저로 설치
- 설치 스크립트 제공

7.3 배포 전략 핵심 요약

Docker 빌드:

- 멀티스테이지 빌드
- Ubuntu 25.04 베이스
- Haswell 타겟으로 최대 호환성
- 테스트 자동화

바이너리 배포:

- monad: 메인 실행 데몬
- monad_cli: CLI 도구
- monad_mpt: TrieDB 관리 도구

성능 튜닝:

- Huge pages 활성화
- io_uring 설정
- CPU 친화성 설정
- NUMA 최적화

7.4 빠른 시작 가이드

1. 의존성 설치:

```
git submodule update --init --recursive
sudo ./scripts/ubuntu-build/install-boost.sh
sudo ./scripts/ubuntu-build/install-tools.sh
sudo ./scripts/ubuntu-build/install-deps.sh
```

2. 빌드:

```
CC=gcc-15 CXX=g++-15 \
CFLAGS="-march=haswell" \
CXXFLAGS="-march=haswell" \
ASMFLAGS="-march=haswell" \
./scripts/configure.sh && ./scripts/build.sh
```

3. 테스트:

```
cd build
CTEST_PARALLEL_LEVEL=$(nproc) ctest
```

4. 실행:

```
./build/cmd/monad  --config  config.json
```

참고 자료

공식 문서

- Monad 하드웨어 요구사항
- Monad 아키텍처

빌드 도구

- CMake 공식 문서
- Ninja 빌드 시스템

컴파일러

- GCC 문서
- Clang 문서

의존성 라이브러리

- Boost 라이브러리
- Intel TBB
- io_uring
- asmjit